

Blocco 5 - JOIN e cardinalità

Combinare relazioni senza duplicare significati

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

Le informazioni utili sono distribuite in più relazioni normalizzate. Per rispondere a una domanda reale bisogna ricomporre clienti, ordini, prodotti e spedizioni senza perdere righe o moltiplicarle in modo non voluto.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Evitare i join copiando dati

Situazione concreta

Una soluzione ingenua è copiare nome cliente e nome prodotto dentro ogni riga ordine. La lettura sembra semplice, ma la ridondanza cresce rapidamente.

ordine	cliente	prodotto	quantità
100	Sara	Sensore A	2
101	Sara	Router B	1
102	Marco	Sensore A	3

Limiti della rappresentazione iniziale

- ▶ duplicazione di clienti e prodotti;
- ▶ aggiornamenti incoerenti;
- ▶ impossibile distinguere dati storici e correnti;
- ▶ aggregazioni rischiano conteggi sbagliati.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Principio

Il join permette di tenere le informazioni nel posto giusto e ricomporle quando servono, usando chiavi primarie ed esterne.

Esempio di riorganizzazione

```
SELECT o.order_id, c.full_name, p.product_name, oi.quantity
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p ON p.product_id = oi.product_id;
```


- ❶ dichiarare la granularità del risultato;
- ❷ scegliere la tabella base;
- ❸ aggiungere tabelle collegate con condizioni di join;
- ❹ decidere se la relazione è obbligatoria o opzionale;
- ❺ controllare il numero di righe prima e dopo.

Analogia: fascicoli collegati da un codice

Analogia o caso esplicativo

In archivio il fascicolo ordine contiene il codice cliente, non copia tutta l'anagrafica. Il join è l'operazione con cui, quando serve, apriamo entrambi i fascicoli e leggiamo i dati collegati tramite quel codice.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

Formalizzazione: JOIN e cardinalità

- ▶ INNER JOIN: conserva solo corrispondenze;
- ▶ LEFT JOIN: conserva la tabella di sinistra;
- ▶ chiave primaria e chiave esterna;
- ▶ cardinalità 1-a-molti;
- ▶ fan-out.

- ▶ **Condizione di join:** regola che abbina righe tra due relazioni.
- ▶ **Join 1-a-molti:** una riga a sinistra può generare molte righe nel risultato.
- ▶ **Fan-out:** moltiplicazione delle righe dopo un join.
- ▶ **Join opzionale:** collegamento che può mancare e richiede `LEFT JOIN`.
- ▶ **Granularità risultato:** una riga per cosa dopo il join?

Come riconoscere una buona soluzione

- ▶ la granularità del risultato è dichiarata;
- ▶ ogni join segue una chiave o una relazione motivata;
- ▶ le righe perse o aggiunte sono spiegate;
- ▶ i controlli di conteggio confermano l'aspettativa.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Domanda: “Mostrare ogni riga ordine con cliente, prodotto, quantità e importo netto.”

Esempio: ragionamento

- ▶ la granularità desiderata è una riga per riga ordine;
- ▶ `order_items` è quindi la tabella base logica;
- ▶ `orders` porta data, stato e cliente;
- ▶ `customers` e `products` aggiungono attributi descrittivi.

Esempio completo: join controllato

```
SELECT o.order_id, c.customer_name, p.sku, p.product_name,  
       oi.quantity,  
       round(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100.0), 2) AS net_amount  
FROM order_items oi  
JOIN orders o ON o.order_id = oi.order_id  
JOIN customers c ON c.customer_id = o.customer_id  
JOIN products p ON p.product_id = oi.product_id  
ORDER BY o.order_id, p.sku;
```

Errori frequenti da prevenire

- ▶ dimenticare una condizione di join;
- ▶ partire dalla tabella sbagliata e cambiare il denominatore;
- ▶ usare `LEFT JOIN` per nascondere un errore di modello;
- ▶ aggregare dopo un join che ha duplicato le righe;
- ▶ non verificare quante righe ci si aspetta.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Esercitazione: problema informale

Contesto

Il team vendite vuole leggere insieme clienti, ordini e prodotti, e vuole anche sapere quali ordini non hanno ancora una spedizione.

Esercitazione: specifica corretta

- ▶ input: tabelle `customers`, `orders`, `order_items`, `products`, `shipments`;
- ▶ output: query con granularità dichiarata e join espliciti;
- ▶ vincolo: usare `INNER JOIN` quando la relazione è obbligatoria;
- ▶ vincolo: usare `LEFT JOIN` solo per collegamenti opzionali come spedizione mancante.

Esercitazione: definizione della soluzione

- ① costruire prima il join clienti-ordini;
- ② aggiungere il dettaglio righe ordine solo quando serve la granularità di dettaglio;
- ③ usare `LEFT JOIN shipments` per evidenziare ordini senza spedizione;
- ④ controllare righe base e righe finali con `count(*)`.

Implementazione completa: ordini senza spedizione (1/2)

```
SET search_path TO training;
```

```
SELECT o.order_id, o.order_date, c.full_name, c.country, o.status
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
ORDER BY o.order_date, o.order_id;
```

```
SELECT o.order_id, c.full_name, p.sku, p.product_name,
       oi.quantity,
       round(oi.quantity * oi.unit_price * (1 - oi.discount_pct / 100.0), 2) AS net_amount
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p ON p.product_id = oi.product_id
ORDER BY o.order_id, p.sku;
```


Implementazione completa: ordini senza spedizione (2/2)

```
SELECT o.order_id, c.full_name, o.order_date, o.status,  
       s.shipment_id, s.shipped_at, s.delivered_at  
FROM orders o  
JOIN customers c ON c.customer_id = o.customer_id  
LEFT JOIN shipments s ON s.order_id = o.order_id  
WHERE s.shipment_id IS NULL  
ORDER BY o.order_date, o.order_id;
```

```
SELECT o.order_id, count(oi.product_id) AS line_rows  
FROM orders o  
LEFT JOIN order_items oi ON oi.order_id = o.order_id  
GROUP BY o.order_id  
ORDER BY line_rows DESC, o.order_id;
```

Esercitazione: organizzazione dei 90 minuti

- ▶ 15 min lettura delle chiavi nello schema;
- ▶ 25 min join obbligatori clienti-ordini-righe;
- ▶ 25 min join opzionali con spedizioni e ticket;
- ▶ 15 min controlli di cardinalità;
- ▶ 10 min discussione fan-out.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ prima del join bisogna sapere una riga per cosa si vuole ottenere;
- ▶ LEFT JOIN non significa “join migliore”: significa relazione opzionale;
- ▶ ogni aggregazione dopo join richiede controllo del fan-out.